

DSA REVISION BIBLE

C-CAT Section B — Complete Study Sheet

Zero to Exam-Ready in 1–2 Hours

8 Topics	50+ Concepts	All Complexities	Exam Traps
Fully Covered	Key Facts Only	Best/Avg/Worst	Highlighted

Prepared for C-CAT 2024–25 • Section B: Data Structures & Algorithms

TABLE OF CONTENTS

Ch 01	Algorithm Complexity & Big-O Big-O ranking, growth order, identifying complexity
Ch 02	Sorting Algorithms Bubble, Selection, Insertion, Merge, Quick, Heap Sort
Ch 03	Searching Algorithms Linear Search, Binary Search — complexity & conditions
Ch 04	Stack Operations, applications, expression conversion & evaluation
Ch 05	Queue Types, operations, circular queue, priority queue
Ch 06	Linked List Types, operations, Floyd's cycle detection
Ch 07	Trees BST, AVL, Heap, traversals — Inorder/Preorder/Postorder
Ch 08	Graphs BFS, DFS, adjacency matrix vs list, MST algorithms
Ch BONUS	Algorithm Paradigms Greedy, Divide & Conquer, Dynamic Programming — key examples

Chapter 01: Algorithm Complexity & Big-O

1.1 Big-O Growth Order (slowest → fastest)

Big-O describes how runtime grows as input size (n) increases. Memorise this order — C-CAT asks you to rank or identify complexities directly.

Notation	Name	Example Algorithm / Operation
$O(1)$	Constant	Array index access, Hash table lookup (avg)
$O(\log n)$	Logarithmic	Binary Search, BST search (avg)
$O(n)$	Linear	Linear Search, single loop over n elements
$O(n \log n)$	Linearithmic	Merge Sort, Heap Sort, Quick Sort (avg)
$O(n^2)$	Quadratic	Bubble / Selection / Insertion Sort (worst)
$O(2^n)$	Exponential	Recursive Fibonacci, Power Set
$O(n!)$	Factorial	Permutation generation (all arrangements)

1.2 Common Operation Complexities

Operation	Time Complexity	Notes
Array: access by index	$O(1)$	Direct jump — no traversal
Array: insert at beginning	$O(n)$	Must shift all elements right
Array: insert at end	$O(1)$	Just place at next slot
Linear Search (unsorted)	$O(n)$	Must check every element
Binary Search (sorted)	$O(\log n)$	Halves search space each step
BST Search (balanced)	$O(\log n)$	Height = $\log n$
BST Search (skewed)	$O(n)$	Degrades to linked list
Hash Table Search (avg)	$O(1)$	Direct index via hash function
Hash Table Search (worst)	$O(n)$	All keys collide → linked list
Recursive Fibonacci	$O(2^n)$	Each call branches into 2 more

1.3 How to Identify Complexity from Code

Pattern	Complexity	Example
Single loop 0 to n	$O(n)$	for i in range(n)
Two nested loops (both n)	$O(n^2)$	for i : for j (both n)
Loop halves each iteration	$O(\log n)$	Binary Search, $i = i*2$

Pattern	Complexity	Example
Loop + inner halving	$O(n \log n)$	Merge Sort merge step
Three nested loops	$O(n^3)$	Matrix multiplication
Recursive $T(n)=2T(n/2)+n$	$O(n \log n)$	Merge Sort recurrence
Recursive $T(n)=T(n-1)+1$	$O(n)$	Simple recursion

■ EXAM TRAP: $O(n \log n)$ is NOT $O(n^2)$. Merge Sort = $O(n \log n)$. Always.

■ EXAM TRAP: Recursive Fibonacci = $O(2^n)$, NOT $O(n)$. Each call creates 2 more branches.

■ EXAM TRAP: Binary Search best case = $O(1)$ (found at mid first try). Average/Worst = $O(\log n)$.

Chapter 02: Sorting Algorithms

2.1 Master Complexity Table

This table is the most exam-critical reference for sorting. Every cell has been tested in C-CAT style MCQs.

Algorithm	Best	Average	Worst	Space	Stable?	In-Place?
Bubble Sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	✓ Yes	✓ Yes
Selection Sort	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$	✗ No	✓ Yes
Insertion Sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	✓ Yes	✓ Yes
Merge Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$	✓ Yes	✗ No
Quick Sort	$O(n \log n)$	$O(n \log n)$	$O(n^2)$	$O(\log n)$	✗ No	✓ Yes
Heap Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(1)$	✗ No	✓ Yes
Counting Sort	$O(n+k)$	$O(n+k)$	$O(n+k)$	$O(k)$	✓ Yes	✗ No

k = range of input values (for Counting Sort) • Quick Sort space = recursion stack depth

2.2 Key Concepts — Stable & In-Place Defined

Term	Plain Language Definition	Example
Stable	Equal elements keep their original relative order after sorting.	Flag(50), Priya(50) → stays Raj, Priya after sort
In-Place	Sorting happens within the original array. No extra array or swaps with $O(1)$ space.	Bubble Sort swaps with $O(1)$ space — no new array
NOT In-Place	Needs an extra array to do its work. Extra space = $O(n)$.	Merge Sort creates a temp array to merge two halves

2.3 Algorithm Behaviour Snapshots

Bubble Sort	Compares and swaps adjacent elements repeatedly. Each pass bubbles the largest unsorted element to its correct position at the end.	Best case $O(n)$ ONLY when early-exit flag detects already-sorted array. Without flag, always $O(n^2)$.
Selection Sort	Finds the minimum element and places it at the correct position. Scans the full remaining array every pass — no early exit possible.	ALWAYS $O(n^2)$ — even on a sorted array. No optimisation possible.
Insertion Sort	Picks each element and inserts it into the correct position in the already-sorted left portion. Think: sorting playing cards in hand.	Best case $O(n)$ = already sorted array needs only n comparisons, 0 swaps. Ideal for small or nearly-sorted arrays.
Merge Sort	Divide & Conquer. Splits array in half recursively, sorts each half, then merges. Always $O(n \log n)$ — no best/worst variation.	Uses $O(n)$ extra space for merge step. Preferred for Linked Lists and external sorting (large files on disk).

Quick Sort	Divide & Conquer. Picks a pivot, partitions array around it, recurses on both sides. Fastest in practice due to cache efficiency.	<i>Worst case $O(n^2)$ when pivot is always the smallest or largest element (already-sorted array with first/last element as pivot).</i>
Heap Sort	Builds a Max-Heap, then repeatedly extracts the maximum element. Always $O(n \log n)$, in-place, but not stable.	<i>Slower than Quick Sort in practice due to poor cache behaviour. $O(1)$ space unlike Merge Sort.</i>

2.4 When to Use Which Sort

Situation	Best Choice	Reason
Need guaranteed $O(n \log n)$	Merge Sort	Quick Sort risks $O(n^2)$
Memory is very limited	Heap Sort	$O(1)$ space, always $O(n \log n)$
Stability required	Merge Sort / Insertion Sort	Both stable
Array is nearly sorted	Insertion Sort	$O(n)$ best case
General purpose, fastest avg	Quick Sort	Cache efficient
Sorting a Linked List	Merge Sort	No random access needed
Small array (< 20 elements)	Insertion Sort	Low overhead

■ **EXAM TRAP:** Merge Sort is $O(n \log n)$ always — no best/worst variation. Quick Sort DOES have variation (worst = $O(n^2)$).

■ **EXAM TRAP:** Selection Sort is ALWAYS $O(n^2)$ — even on a sorted array. No early exit possible.

■ **EXAM TRAP:** Quick Sort worst case = already sorted array with first or last element chosen as pivot.

■ **EXAM TRAP:** Heap Sort and Merge Sort both guarantee $O(n \log n)$ worst case — but Heap Sort is in-place ($O(1)$ space); Merge Sort is not ($O(n)$ space).

■ **EXAM TRAP:** Only Insertion Sort is BOTH stable AND in-place among the comparison sorts listed.

Chapter 03: Searching Algorithms

3.1 Complexity Comparison

Algorithm	Best	Average	Worst	Prerequisite	Data Structure
Linear Search	$O(1)$	$O(n)$	$O(n)$	None — works on any array	Array / Linked List
Binary Search	$O(1)$	$O(\log n)$	$O(\log n)$	Array MUST be sorted	Sorted Array only

3.2 How Binary Search Works

Step 1: Check the middle element.

Step 2: If target = mid → found. Return index.

Step 3: If target < mid → search LEFT half.

Step 4: If target > mid → search RIGHT half.

Step 5: Repeat until found or range is empty → not found.

3.3 Binary Search — Simulation Example

Step	Array State	Action	Result
Start	[2, 5, 8, 12, 16, 23, 38] Target=23	mid = index 3 = 12	23 > 12 → go RIGHT
Step 2	[16, 23, 38]	mid = index 5 = 23	23 = 23 → FOUND at index 5

■ **EXAM TRAP:** Binary Search **REQUIRES** a sorted array. On unsorted input, it gives **WRONG** results — not just slow results.

■ **EXAM TRAP:** Binary Search best case = $O(1)$ when target is exactly at the middle on first check.

Chapter 04: Stack

4.1 Core Concept

Stack = LIFO (Last In, First Out). The last element pushed is the first one popped. Think of a stack of plates — you add and remove from the top only.

4.2 Operation Complexities

Operation	Time Complexity	Space	Description
Push	$O(1)$	$O(1)$	Add element to top
Pop	$O(1)$	$O(1)$	Remove element from top
Peek / Top	$O(1)$	$O(1)$	View top element without removing
Search	$O(n)$	$O(1)$	Must traverse — no direct access
isEmpty	$O(1)$	$O(1)$	Check if stack has no elements
Total Stack Space	—	$O(n)$	n elements stored in stack

4.3 Applications of Stack

Application	Why Stack?
Infix → Postfix / Prefix conversion	Handles operator precedence using LIFO ordering
Postfix expression evaluation	Operands pushed; operator pops two and pushes result
Balancing parentheses / brackets	Push open brackets; pop and match on close bracket
Function call management	Each function call pushed; returns pop the call stack
Undo operation (editors)	Last action is undone first — LIFO
Recursion (internally)	Each recursive call is pushed onto the system call stack
DFS traversal of Graph	Uses stack to track vertices to visit next
Browser back button	Previous pages stored in stack — most recent first

4.4 Expression Notations

Notation	Operator Position	Example	Used By
Infix	Between operands	$A + B$	Humans — natural math writing
Postfix (RPN)	After operands	$A B +$	Computers — easy to evaluate
Prefix (Polish)	Before operands	$+ A B$	Some compilers

4.5 Infix → Postfix Conversion Rules

Rule 1 — Operand (A, B, numbers): Write directly to OUTPUT. Always. No thinking.

Rule 2 — Opening bracket '(': Push onto STACK. Always.

Rule 3 — Closing bracket ')': Pop from STACK and write to output until '(' found. Discard both brackets.

Rule 4 — Operator (+, -, *, /): If stack empty → push. If top has equal/higher precedence → pop top to output first, then push current. If top has lower precedence → just push.

End of expression: Pop ALL remaining operators from stack to output.

Precedence: ^ (highest) > * / > + - (lowest)

4.6 Infix → Postfix — Worked Examples

Infix Expression	Postfix Result	Key Step
A + B * C	A B C * +	* has higher precedence → processed before +
(A + B) * C	A B + C *	Brackets force A+B first
(A + B) * (C - D)	A B + C D - *	Two bracket groups, then *
A + B + C	A B + C +	Left to right for same precedence

4.7 Postfix Evaluation Rules

Rule 1 — Operand (number): PUSH onto stack.

Rule 2 — Operator: POP two operands. Apply operator. PUSH result back.

Order matters: First popped = right operand. Second popped = left operand.

Final answer: Last remaining value on stack.

Postfix	Step-by-step	Result
2 3 4 * +	Push 2, Push 3, Push 4 → * pops 3,4 → 12 → Push 12 → + pops 2,12 → 14	14
5 1 2 + 4 * + 3 -	Push 5,1,2 → + pops 1,2=3 → Push 3 → Push 4 → * pops 3,4=12 → + pops 5,12=17 → Push 17 → - = 14	

■ **EXAM TRAP:** Operands ALWAYS go straight to output in conversion. NEVER push an operand onto the stack.

■ **EXAM TRAP:** When you see ')' — pop until '(' but do NOT write brackets in the output. Brackets disappear.

Chapter 05: Queue

5.1 Core Concept

Queue = FIFO (First In, First Out). The first element enqueued is the first one dequeued. Think of a ticket counter line — first person in line is served first.

5.2 Operation Complexities

Operation	Simple Queue	Circular Queue	Notes
Enqueue (add)	$O(1)$	$O(1)$	Add at rear
Dequeue (remove)	$O(1)$	$O(1)$	Remove from front
Peek / Front	$O(1)$	$O(1)$	View front without removing
Search	$O(n)$	$O(n)$	Must traverse — no direct access
isEmpty	$O(1)$	$O(1)$	Check front == rear

5.3 Types of Queue

Type	Key Property	Use Case
Simple (Linear) Queue	FIFO. Fixed front and rear. Rear cannot wrap around.	Basic task scheduling
Circular Queue	Rear wraps to front when end is reached. Reuses empty slots. Solves wasted space problem of simple queue.	CPU Scheduling, buffering
Priority Queue	Element with HIGHEST priority dequeues first. NOT purely FIFO.	Dijkstra's algorithm, OS scheduling
Deque (Double-ended Queue)	Insert AND delete from BOTH front and rear ends.	Sliding window problems, palindrome checking
Input Restricted Deque	Insert at rear only. Delete from both ends.	Specialised scheduling
Output Restricted Deque	Delete from front only. Insert from both ends.	Specialised buffering

5.4 Applications of Queue

Application	Why Queue?
BFS traversal of Graph	Processes nodes level by level — FIFO order
CPU process scheduling	Processes served in arrival order
Printer spooling	Print jobs processed in order received
Keyboard buffer	Keystrokes processed in order typed
Call center systems	First caller is served first

■ **EXAM TRAP:** Stack = LIFO. Queue = FIFO. C-CAT describes behaviour and asks which structure — don't confuse under pressure.

■ EXAM TRAP: Circular Queue does NOT increase array size. It reuses empty slots at the front. Same fixed size, smarter use.

■ EXAM TRAP: Priority Queue is NOT a simple FIFO queue. Order depends on priority, not arrival time.

Chapter 06: Linked List

6.1 What is a Node?

Singly LL Node: [DATA | NEXT →] — one pointer to next node
Doubly LL Node: [← PREV | DATA | NEXT →] — two pointers
Last node NEXT: Points to NULL (non-circular) or HEAD (circular)
Head pointer: Always points to the FIRST node

6.2 Types of Linked List

Type	Direction	Last Node Points To	Extra Memory
Singly Linked List	One-way (→)	NULL	1 pointer per node
Doubly Linked List	Both ways (← →)	NULL	2 pointers per node
Circular Singly	One-way (→)	HEAD (first node)	1 pointer per node
Circular Doubly	Both ways (← →)	HEAD (first node)	2 pointers per node

6.3 Operation Complexities

Operation	Singly LL	Doubly LL	Array (comparison)
Access by index	$O(n)$	$O(n)$	$O(1)$
Search	$O(n)$	$O(n)$	$O(n)$
Insert at beginning	$O(1)$	$O(1)$	$O(n)$
Insert at end	$O(n)^*$	$O(n)^*$	$O(1)$
Insert at middle	$O(n)$	$O(n)$	$O(n)$
Delete at beginning	$O(1)$	$O(1)$	$O(n)$
Delete at end	$O(n)$	$O(n)$	$O(1)$
Delete (given node ptr only)	$O(n)$	$O(1)$	—

* $O(1)$ if tail pointer is maintained separately

6.4 Singly vs Doubly — Key Trade-offs

Property	Singly LL	Doubly LL
Pointers per node	1 (next only)	2 (next + prev)
Memory per node	Less	More
Reverse traversal	Not possible	Possible

Property	Singly LL	Doubly LL
Delete (given only node ptr)	$O(n)$ — need previous node	$O(1)$ — use prev pointer
Implementation complexity	Simpler	More complex

6.5 Linked List vs Array — When to Use Which

Situation	Use	Reason
Frequent insert/delete at beginning	Linked List	$O(1)$ vs $O(n)$ for array
Frequent access by index	Array	$O(1)$ vs $O(n)$ for LL
Size changes dynamically	Linked List	No reallocation needed
Memory is fixed and known in advance	Array	Less overhead per element
Random access needed	Array	No direct index in LL
Sorting (Merge Sort preferred)	Linked List	No random access required

6.6 Floyd's Cycle Detection Algorithm

Purpose: Detect if a linked list has a loop/cycle.

Two pointers used: SLOW (moves 1 step at a time) and FAST (moves 2 steps at a time).

Cycle exists: If SLOW and FAST pointers meet at the same node.

No cycle: If FAST pointer reaches NULL.

Time complexity: $O(n)$ | **Space complexity:** $O(1)$

6.7 Node Insertion — Step Order Matters

Insert 25 between 20 and 30 in: HEAD → 10 → 20 → 30 → NULL

Step 1 (FIRST): Set 25's NEXT = 30 (point new node to next node FIRST)

Step 2 (SECOND): Set 20's NEXT = 25 (point previous node to new node)

If you do Step 2 first, you lose the reference to 30 — the list breaks!

■ **EXAM TRAP:** Accessing element in LL = $O(n)$. No index jumping like arrays. Must traverse from HEAD every time.

■ **EXAM TRAP:** Insert at BEGINNING = $O(1)$. Insert at END = $O(n)$. Students often assume both are $O(1)$.

■ **EXAM TRAP:** Circular LL: last node points to FIRST node, NOT NULL. Don't write NULL by habit.

■ **EXAM TRAP:** Floyd's needs exactly TWO pointers — slow and fast. Not one, not three.

Chapter 07: Trees

7.1 Core Terminology

Term	Plain Language Definition
Root	Top node of the tree. Has no parent.
Parent	A node that has at least one child below it.
Child	A node that comes directly below a parent.
Leaf	A node with NO children. Bottom-most nodes.
Height	Longest path from root to any leaf. Root alone = height 0.
Depth	Distance of a node from the root. Root depth = 0.
Degree	Number of children a node has.
Subtree	A node and all its descendants.

7.2 Types of Trees

Type	Key Rule	C-CAT Focus
Binary Tree	Each node has AT MOST 2 children.	Foundation for BST, Heap
Full Binary Tree	Every node has 0 or 2 children. No node has 1 child.	Structure questions
Complete Binary Tree	All levels full except possibly last; last level filled left to right.	Used in Heap
Binary Search Tree (BST)	Left subtree < Node < Right subtree. Always.	Search, Insert, Delete
AVL Tree	BST + self-balancing. Balance factor of every node = -1, 0, or 1.	Guaranteed $O(\log n)$
Heap	Complete binary tree. Max-Heap: parent $\geq$ children. Min-Heap: parent $\leq$ children.	Heap Sort, Priority Queue

7.3 BST Complexities

Operation	Average Case (Balanced)	Worst Case (Skewed)
Search	$O(\log n)$	$O(n)$
Insert	$O(\log n)$	$O(n)$
Delete	$O(\log n)$	$O(n)$
Inorder Traversal	$O(n)$	$O(n)$

Skewed BST: All nodes go to one side — looks like a linked list. Example: inserting 10, 20, 30, 40 in order creates a right-skewed BST. Search = $O(n)$.

7.4 Tree Traversals — The Big Three

Traversal	Order	Memory Trick	Output on Sample Tree
Inorder	Left → Root → Right	LRR (Left, Root, Right)	4, 6, 8, 10, 15, 20
Preorder	Root → Left → Right	RoLR (Root, Left, Right)	10, 6, 4, 8, 15, 20
Postorder	Left → Right → Root	LRRL (Left, Right, Root)	4, 8, 6, 20, 15, 10
Level Order (BFS)	Level by level, left to right	Use a Queue	10, 6, 15, 4, 8, 20

Sample tree used: Root=10, Left subtree: 6(4,8), Right subtree: 15(┐,20)

7.5 AVL Tree

Balance Factor = Height(Left Subtree) - Height(Right Subtree)

Allowed values: -1, 0, or +1 only. Any other value triggers a rotation.

Rotations: LL Rotation, RR Rotation, LR Rotation, RL Rotation

Guarantee: All operations (search, insert, delete) = $O(\log n)$ ALWAYS

Key difference from BST: BST can degrade to $O(n)$. AVL cannot.

7.6 Heap

Operation	Time Complexity	Notes
Get Max (Max-Heap)	$O(1)$	Root is always the maximum
Get Min (Min-Heap)	$O(1)$	Root is always the minimum
Insert	$O(\log n)$	Insert at end, bubble up (heapify up)
Delete Max/Min	$O(\log n)$	Replace root with last, bubble down (heapify down)
Build Heap from array	$O(n)$	NOT $O(n \log n)$ — important trap

Max-Heap rule: Every parent $\geq$ its children. Root = largest element.

Min-Heap rule: Every parent $\leq$ its children. Root = smallest element.

Heap is NOT a BST — no left $<$ root $<$ right rule. Only parent-child relationship matters.

■ **EXAM TRAP:** Inorder traversal of a BST ALWAYS gives sorted ascending output. This is a direct C-CAT fact.

■ **EXAM TRAP:** AVL Tree IS a BST — it has ALL BST properties PLUS self-balancing. Not a separate structure.

■ **EXAM TRAP:** Heap is NOT a BST. No left $<$ root $<$ right rule applies in a Heap.

■ **EXAM TRAP:** BST worst case = $O(n)$ when skewed. AVL Tree worst case = $O(\log n)$ always. C-CAT tests this difference directly.

■ **EXAM TRAP:** Building a Heap = $O(n)$, NOT $O(n \log n)$. This surprises most students.

Chapter 08: Graphs

8.1 Core Terminology

Term	Definition
Vertex (Node)	A point/entity in the graph.
Edge	A connection between two vertices.
Degree	Number of edges connected to a vertex.
Path	Sequence of vertices connected by edges.
Cycle	A path that starts and ends at the same vertex.
Connected Graph	Every vertex is reachable from every other vertex.
Disconnected Graph	Some vertices cannot be reached from others.
DAG	Directed Acyclic Graph — directed edges, no cycles.

8.2 Types of Graphs

Type	Key Property
Undirected	Edges have no direction. A-B means travel both ways.
Directed (Digraph)	Edges have direction. A→B means one way only.
Weighted	Edges have a cost/weight (e.g., distance, time).
Unweighted	Edges have no cost — just connected or not.
Cyclic	Contains at least one cycle.
Acyclic	No cycles. Trees are a special case of acyclic graphs.

8.3 Graph Representation

Property	Adjacency Matrix	Adjacency List
Storage structure	2D array $V \times V$. $matrix[i][j]=1$ if edge exists.	Each vertex stores list of its neighbours.
Space complexity	$O(V^2)$	$O(V + E)$
Check if edge exists	$O(1)$	$O(\text{degree of vertex})$
Find all neighbours	$O(V)$	$O(\text{degree of vertex})$
Best for	Dense graphs (many edges)	Sparse graphs (few edges)
Memory efficiency	Poor for sparse graphs	Efficient for sparse graphs

8.4 BFS vs DFS — The Core Comparison

Property	BFS (Breadth First)	DFS (Depth First)
Data structure used	Queue	Stack (or recursion)
Traversal style	Level by level	Branch by branch (go deep first)
Finds shortest path	Yes (unweighted graphs)	No
Memory usage	More (stores all neighbours of current level)	Less
Time complexity	$O(V + E)$	$O(V + E)$
Space complexity	$O(V)$	$O(V)$
Good for	Shortest path, nearby nodes, level-order	Cycle detection, maze solving, topological sort
Internal mechanism	Enqueue neighbours, dequeue to visit	Push to stack, pop to visit (or recurse)

8.5 Shortest Path Algorithms

Algorithm	Graph Type	Handles Negative Weights	Time Complexity
BFS	Unweighted only	N/A	$O(V + E)$
Dijkstra's	Weighted, no negative edges	No	$O(E \log V)$
Bellman-Ford	Weighted, any edges	Yes	$O(VE)$
Floyd-Warshall	All pairs shortest path	Yes	$O(V^3)$

8.6 Minimum Spanning Tree (MST) Algorithms

MST: A subset of edges that connects ALL vertices with MINIMUM total weight and NO cycles.

Number of edges in MST: Always $V-1$ (V = number of vertices)

Algorithm	Approach	How It Works	Time Complexity	Best For
Kruskal's	Greedy	Sort all edges by weight. Add smallest edge that doesn't create a cycle. Stop at $V-1$ edges.	$O(E \log E)$	Sparse graphs.
Prim's	Greedy	Start from one vertex. Always add cheapest edge connecting a vertex to a new vertex.	$O(E \log V)$	Dense graphs

■ **EXAM TRAP:** BFS uses a QUEUE. DFS uses a STACK. C-CAT swaps these in wrong options — memorise this cold.

■ **EXAM TRAP:** BFS finds shortest path ONLY in UNWEIGHTED graphs. For weighted graphs, use Dijkstra's.

■ **EXAM TRAP:** Adjacency Matrix space = $O(V^2)$. Adjacency List space = $O(V+E)$. For sparse graphs, List is far more efficient.

■ **EXAM TRAP:** Both BFS and DFS have SAME time complexity $O(V+E)$. Neither is inherently faster than the other.

■ **EXAM TRAP:** Kruskal's AND Prim's are both GREEDY algorithms. Never Dynamic Programming or Divide & Conquer.

Chapter BONUS: Algorithm Paradigms

B.1 The Three Main Paradigms

Paradigm	Core Idea	Key Examples
Greedy	Make the locally optimal choice at each step. Never reconsider previous choices.	Kruskal's, Prim's, Dijkstra's, Huffman, Activity Selection
Divide & Conquer	Split problem into smaller subproblems. Solve each recursively, then merge results.	Merge Sort, Quick Sort, Binary Search, Strassen's Matrix Multiplication
Dynamic Programming	Break problem into overlapping subproblems. Store results to avoid recomputation.	Fibonacci, Longest Common Subsequence, Knapsack, Shortest Path

B.2 Divide & Conquer — Three Phases

Phase 1 — DIVIDE: Split the problem into smaller subproblems of the same type.

Phase 2 — CONQUER: Solve each subproblem recursively (base case: solve directly).

Phase 3 — COMBINE: Merge the solutions of subproblems into the final answer.

Conquer = solving the subproblems. Combine = merging their results. These are distinct phases.

B.3 Quick Identification Guide

If question mentions...	The paradigm is...
'locally optimal', 'greedy choice', Kruskal's, Prim's, Dijkstra's, Huffman	Greedy
'split into halves', Merge Sort, Quick Sort, Binary Search	Divide & Conquer
'overlapping subproblems', 'memoization', Fibonacci, Knapsack, DP	Dynamic Programming
'backtracking', 'try all possibilities', N-Queens, Sudoku solver	Backtracking
'explore all neighbours', BFS, level-order	Queue-based traversal
'go deep first', DFS, cycle detection	Stack-based traversal

■ **EXAM TRAP:** Dijkstra's = Greedy. NOT Dynamic Programming, even though it stores distances. Greedy choice = always pick smallest distance vertex.

■ **EXAM TRAP:** Merge Sort and Quick Sort = Divide & Conquer. NOT Greedy. They split, recurse, and combine.

■ **EXAM TRAP:** 'Conquer' phase ≠ 'Combine' phase. Conquer = solving subproblems. Combine = merging their results. C-CAT tests this distinction.

B.4 Master Summary — All Complexities at a Glance

Topic / Algorithm	Best	Average	Worst	Space	Stable	In-Place
Bubble Sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	Yes	Yes
Selection Sort	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$	No	Yes
Insertion Sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	Yes	Yes
Merge Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$	Yes	No
Quick Sort	$O(n \log n)$	$O(n \log n)$	$O(n^2)$	$O(\log n)$	No	Yes
Heap Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(1)$	No	Yes
Linear Search	$O(1)$	$O(n)$	$O(n)$	$O(1)$	—	—
Binary Search	$O(1)$	$O(\log n)$	$O(\log n)$	$O(1)$	—	—
BST Search (balanced)	—	$O(\log n)$	$O(n)$ skewed	$O(n)$	—	—
AVL Search	—	$O(\log n)$	$O(\log n)$	$O(n)$	—	—
Heap Insert/Delete	—	$O(\log n)$	$O(\log n)$	$O(n)$	—	—
BFS / DFS	—	$O(V+E)$	$O(V+E)$	$O(V)$	—	—
Dijkstra's	—	$O(E \log V)$	$O(E \log V)$	$O(V)$	—	—
Kruskal's MST	—	$O(E \log E)$	$O(E \log E)$	$O(V)$	—	—
Stack Push/Pop	$O(1)$	$O(1)$	$O(1)$	$O(1)$	—	—
Queue Enqueue/Dequeue	$O(1)$	$O(1)$	$O(1)$	$O(1)$	—	—